

Universidad Tecnológica de Bolívar

Maestría en Ingeniería

Informe de Ejercicios Propuestos

Asignatura: Deep Learning — Notebooks 07 al 13

Estudiante: Harold Styven Lagares De Voz

Profesor: Jairo Serrano Castañeda

Posgrado: Maestría en Ingeniería

Institución: Universidad Tecnológica de Bolívar

Fecha: Junio de 2026

Este informe sintetiza los resultados y conclusiones de los ejercicios desarrollados en los notebooks 07 a 13 de la asignatura de Deep Learning.

Índice

1. Introducción	2
2. Notebook 07: Transformers y NLP con spaCy	2
2.1. Eficiencia del procesamiento por lotes con <code>nlp.pipe()</code>	2
2.2. Vectores GloVe como análogo a la atención: ranking semántico	2
2.3. Extensión del pipeline NER con <code>EntityRuler</code>	3
2.4. Reflexión	3
3. Notebook 08: Redes Generativas Antagónicas (GANs)	3
3.1. GAN densa en Fashion MNIST	3
3.2. DCGAN con arquitectura convolucional	4
3.3. Reflexión	4
4. Notebook 09: Autoencoders	4
4.1. Exploración del espacio latente del VAE mediante muestreo e interpolación	4
4.2. Autoencoder convolucional: comparativa de calidad de reconstrucción	5
4.3. Reflexión	5
5. Notebook 10: Clustering y Reducción de Dimensionalidad	6
5.1. Evaluación cuantitativa de PCA y t-SNE en el dataset Digits	6
5.2. Clustering jerárquico Ward y dendrograma	6
5.3. Reflexión	7
6. Notebook 11: Interpretabilidad de Modelos	7
6.1. Consistencia de SHAP entre Random Forest y Gradient Boosting	7
6.2. Modelo surrogate para interpretabilidad eficiente de redes neuronales	7
6.3. Reflexión	8
7. Notebook 12: CPU, GPU y Aceleración de Hardware	8
7.1. Impacto del <i>batch size</i> en el tiempo de entrenamiento	8
7.2. Optimización del pipeline de datos con <code>tf.data</code>	8
7.3. Reflexión	9
8. Notebook 13: Despliegue de Modelos	9
8.1. Endpoint de predicción por lotes	9
8.2. Contenedorización multi-etapa con Docker	9
8.3. Reflexión	10
9. Conclusiones Generales	10
Referencias	11

1. Introducción

El presente informe documenta los resultados y conclusiones derivados de la resolución de los ejercicios propuestos en los notebooks 07 a 13 del curso de Deep Learning. Este tramo del recorrido cubre los temas de mayor complejidad técnica: procesamiento de lenguaje natural con vectores semánticos, redes generativas antagónicas, autoencoders variacionales, técnicas de clustering y reducción de dimensionalidad, interpretabilidad de modelos, optimización del cómputo mediante aceleración por hardware, y despliegue de modelos en entornos productivos.

Al igual que en el informe correspondiente a los notebooks 01–06, el objetivo de este documento no es reproducir el código implementado, sino articular de manera formal las observaciones, comparaciones y aprendizajes que emergieron de cada experimento. Las implementaciones aquí documentadas corresponden a la revisión y ampliación del marco de trabajo original, incorporando nuevas técnicas que profundizan los conceptos teóricos de cada tema.

2. Notebook 07: Transformers y NLP con spaCy

2.1 Eficiencia del procesamiento por lotes con `nlp.pipe()`

El primer ejercicio abordó una comparativa de rendimiento entre el procesamiento secuencial de texto mediante bucle simple (`nlp(texto)`) y el procesamiento por lotes mediante `nlp.pipe()`. Se generaron 100 textos a partir de cinco frases base y se midió el tiempo total de procesamiento con el modelo `en_core_web_md` de spaCy (vectores GloVe de 300 dimensiones sobre un vocabulario de 20,000 palabras).

Los resultados mostraron que `nlp.pipe()` con un tamaño de lote de 32 elementos completó el procesamiento en aproximadamente el 55 % del tiempo requerido por el bucle simple, una aceleración de alrededor de 1,8×. La equivalencia de los resultados entre ambos métodos fue verificada mediante una aserción sobre los textos del objeto `Doc` producido. Esta diferencia, moderada en lotes de 100 textos, se amplifica significativamente con volúmenes del orden de decenas de miles, donde el costo fijo de inicialización del pipeline se amortiza en un mayor número de operaciones.

2.2 Vectores GloVe como análogo a la atención: ranking semántico

El tercer ejercicio exploró la capacidad de los vectores de palabras GloVe para ordenar frases por su proximidad semántica a una referencia, reproduciendo conceptualmente la función que ejerce la matriz de atención en los Transformers. Partiendo de la frase “Artificial intelligence is changing the world” como referencia, se calculó la similitud coseno (`doc.similarity()`) entre dicha referencia y ocho frases candidatas de dominios semánticos variados (inteligencia artificial, deportes, clima, mercados financieros).

Las frases del dominio de aprendizaje automático y redes neuronales obtuvieron similitudes superiores a 0.93, mientras que las frases de dominios semánticamente alejados (deportes, entorno doméstico) no superaron el 0.65. La visualización mediante gráfica

de barras horizontales con escala de color `RdYlGn` hizo explícita la gradación de la relevancia semántica. Este ejercicio confirma que la similitud coseno entre embeddings estáticos de frases captura relaciones temáticas de forma análoga a como las cabezas de atención en BERT ponderan la relevancia entre pares de tokens.

2.3 Extensión del pipeline NER con `EntityRuler`

El cuarto ejercicio incorporó un componente `EntityRuler` personalizado al pipeline de spaCy, insertado antes del reconocedor de entidades por defecto para garantizar precedencia. Se definieron 14 patrones distribuidos en dos etiquetas: `AI_MODEL` (modelos de lenguaje e imagen como GPT-4, BERT, Claude, Stable Diffusion) y `AI_ORG` (organizaciones como OpenAI, Anthropic, DeepMind, Hugging Face).

Al procesar cuatro oraciones de prueba sobre el ecosistema de la inteligencia artificial, el pipeline extendido detectó correctamente la totalidad de las entidades personalizadas sin interferir con las entidades estándar del modelo base (personas, lugares, fechas). La visualización mediante `displaCy` con colores diferenciados (`#4fc3f7` para modelos, `#81c784` para organizaciones) permitió verificar la correcta segmentación. La combinación del `EntityRuler` con el NER preentrenado representa una estrategia de adaptación de dominio sin re-entrenamiento, particularmente valiosa cuando el vocabulario especializado del dominio es estable y acotado.

2.4 Reflexión

El uso de spaCy con vectores GloVe preentrenados ofrece una alternativa robusta al ecosistema HuggingFace para tareas de NLP de mediana complejidad: sin requerimiento de token de autenticación, con velocidad de inferencia superior y con capacidad de extensión mediante reglas explícitas. La combinación de embeddings estáticos para similitud semántica y el `EntityRuler` para reconocimiento de entidades de dominio cubre el 80 % de los casos de uso industriales más frecuentes en producción.

3. Notebook 08: Redes Generativas Antagónicas (GANs)

3.1 GAN densa en Fashion MNIST

El segundo ejercicio evaluó la capacidad de generalización de la arquitectura GAN basada en capas densas al dataset Fashion MNIST, compuesto por 60,000 imágenes de 28×28 píxeles de prendas de vestir en escala de grises. La misma arquitectura empleada para MNIST (generador con `BatchNormalization` y `LeakyReLU`, label smoothing en 0.9) fue entrenada durante 1,000 épocas con `batch_size` de 128 y un optimizador Adam con tasa de aprendizaje de 10^{-4} .

El modelo logró producir siluetas reconocibles de prendas de ropa, aunque con el nivel de borrosidad característico de las arquitecturas densas. El equilibrio entre las pérdidas del discriminador y el generador fue más inestable que en MNIST, con la pérdida del discriminador tendiendo a crecer progresivamente a partir de la época 400, señal de

que el generador no logró mantener el ritmo de aprendizaje del discriminador ante la mayor complejidad espacial de las prendas. Fashion MNIST, al presentar variaciones texturales sutiles (costuras, pliegues, contornos irregulares), requiere que el modelo capture dependencias locales de píxeles que las capas densas, al operar sobre vectores aplanados de 784 elementos, no pueden modelar eficientemente.

3.2 DCGAN con arquitectura convolucional

El tercer ejercicio implementó una DCGAN (*Deep Convolutional GAN*) sobre MNIST, sustituyendo las capas densas por una arquitectura jerárquica de convoluciones. El generador parte de un vector de ruido de dimensión 100 y lo proyecta a un tensor de $7 \times 7 \times 128$, que es progresivamente expandido mediante dos capas `Conv2DTranspose` con *strides* de 2 hasta alcanzar la resolución objetivo de $28 \times 28 \times 1$. El discriminador aplica la transformación inversa con `Conv2D` con *strides*, reduciendo la imagen a un escalar de probabilidad.

La comparativa directa entre la GAN densa y la DCGAN, alimentadas con el mismo vector de ruido, evidenció una mejora cualitativa sustancial: la DCGAN produjo trazos con bordes mejor definidos, mayor contraste entre el trazo y el fondo, y menor presencia de artefactos de alta frecuencia. El uso de hiperparámetros consagrados para la estabilidad adversarial (`LeakyReLU` con pendiente 0.2, `BatchNormalization`, Adam con $\beta_1 = 0.5$) mantuvo las pérdidas del discriminador y el generador en rangos comparables a lo largo de las 1,000 épocas de entrenamiento.

3.3 Reflexión

La elección de arquitectura en las GANs tiene un impacto directo y observable en la calidad generativa, no solo en las métricas de pérdida. Las convoluciones transpuestas no son simplemente una variante más compleja, sino la herramienta apropiada cuando el dominio del problema tiene estructura espacial. Para imágenes de resolución superior a 64×64 píxeles, las DCGAN y sus descendientes (StyleGAN, Diffusion Models) son la única opción práctica.

4. Notebook 09: Autoencoders

4.1 Exploración del espacio latente del VAE mediante muestreo e interpolación

El segundo ejercicio aprovechó el VAE entrenado (espacio latente bidimensional) para explorar de forma sistemática la continuidad y la estructura de la distribución aprendida. Se realizaron dos experimentos complementarios: un muestreo aleatorio desde $\mathcal{N}(0, I)$ (15 puntos) y una interpolación en cuadrícula de 15×15 sobre la región $[-3, 3]^2$ del espacio latente.

El muestreo aleatorio produjo imágenes de dígitos reconocibles pero con cierta variabilidad de calidad, esperada dada la distancia variable de los puntos muestreados respecto a

las regiones de alta densidad de la distribución posterior. El experimento más revelador fue la cuadrícula de interpolación: al recorrer el espacio latente de forma sistemática, las transiciones entre dígitos son suaves y continuas, sin discontinuidades abruptas. Las zonas centrales del espacio concentran los dígitos de mayor frecuencia (1, 7), mientras que las zonas periféricas contienen dígitos menos representados o formas de transición. Este comportamiento confirma que la regularización KL cumplió su función: el espacio latente aprendido es continuo y muestreable, no una colección de islas discretas como ocurriría en un autoencoder estándar.

4.2 Autoencoder convolucional: comparativa de calidad de reconstrucción

El tercer ejercicio implementó un autoencoder convolucional con una arquitectura encoder-decoder completamente simétrica: el encoder aplica dos bloques **Conv2D** + **MaxPooling2D** (de 28×28 a 14×14 a 7×7), mientras que el decoder los invierte mediante **Conv2DTranspose** + **UpSampling2D**. El espacio latente resultante tiene dimensión $7 \times 7 \times 16$ (784 valores), equiparable al `encoding_dim = 32` del autoencoder denso.

La Tabla 1 compara los cuatro modelos de autoencoder entrenados sobre MNIST en términos de MSE en el conjunto de prueba.

Cuadro 1: Comparativa de error de reconstrucción (MSE) entre variantes de autoencoder.

Modelo	MSE test	Espacio latente	Genera datos	Tiempo (s)
AE Denso	$\approx 0,00123$	32 dims	No	≈ 8
Denoising AE	$\approx 0,00116$	32 dims	No	≈ 5
VAE	$\approx 0,00342$	2 dims	Sí	≈ 20
AE Convolucional	$\approx 0,00082$	$7 \times 7 \times 16$	No	≈ 18

El autoencoder convolucional obtuvo el MSE más bajo de los cuatro modelos. La diferencia respecto al VAE no debe interpretarse como una superioridad incondicional: el VAE sacrifica precisión de reconstrucción para ganar estructura latente muestreable, un intercambio explícitamente regulado por la pérdida KL. Entre el autoencoder denso y el convolucional, la diferencia de aproximadamente un tercio en el MSE se explica por la capacidad de las convoluciones para capturar dependencias espaciales locales (bordes, curvas de los dígitos) que los pesos densos, operando sobre vectores aplanados, no pueden modelar sin recurrir a una mayor capacidad paramétrica.

4.3 Reflexión

La familia de autoencoders ilustra con claridad el principio de diseño orientado al objetivo: la arquitectura más adecuada no es la de menor MSE, sino la que mejor se alinea con el uso esperado del modelo. La elección entre autoencoder denso, denoising, variacional o convolucional debe basarse en si el objetivo es compresión eficiente, robustez al ruido, generación controlada o máxima fidelidad de reconstrucción, respectivamente.

5. Notebook 10: Clustering y Reducción de Dimensionalidad

5.1 Evaluación cuantitativa de PCA y t-SNE en el dataset Digits

El tercer ejercicio extendió el análisis de reducción de dimensionalidad al dataset *Digits* de scikit-learn (1,797 muestras, 64 *features*, 10 clases), un escenario más exigente que Iris en cuanto a dimensionalidad y número de clases. Se aplicaron PCA (reducción lineal directa a 2 componentes) y t-SNE (con pre-reducción a 50 componentes mediante PCA para acelerar el cómputo). Sobre cada proyección se aplicó K-Means con $K = 10$ y se calcularon el *Adjusted Rand Index* (ARI) y el *Silhouette Score*.

PCA completó la proyección en menos de 0.05 segundos, explicando apenas el 28.6 % de la varianza total en las dos primeras componentes. La visualización resultante mostró solapamiento considerable entre clases, particularmente entre los dígitos 3, 5, 8 y 9. El K-Means aplicado sobre la proyección PCA obtuvo un ARI de aproximadamente 0.41. t-SNE requirió cerca de 12 segundos pero logró separar las 10 clases en agrupaciones visualmente distintas, con el K-Means sobre la proyección t-SNE alcanzando un ARI de aproximadamente 0.68. Sin embargo, el K-Means aplicado directamente sobre los 64 *features* escalados superó a ambas proyecciones con el mayor ARI (0.72), lo que evidencia una pérdida de información métrica global durante la reducción.

5.2 Clustering jerárquico Ward y dendrograma

El cuarto ejercicio implementó `AgglomerativeClustering` con criterio de enlace Ward sobre el dataset Iris, y construyó el dendrograma correspondiente sobre una muestra de 50 observaciones mediante la función `linkage` de SciPy. El dendrograma, cortado a una altura de 3.5 unidades de distancia Ward, sugiere naturalmente la presencia de 3 grupos, coincidiendo con las 3 especies reales del dataset.

La comparativa de los tres métodos aplicados a Iris se resume en la Tabla 2.

Cuadro 2: Comparativa de métodos de clustering sobre el dataset Iris.

Método	Silhouette Score	ARI
K-Means ($K = 3$)	0.551	0.730
DBSCAN ($\varepsilon = 0.6$)	0.489	0.562
Jerárquico Ward ($K = 3$)	0.547	0.721

K-Means y el método jerárquico Ward obtuvieron rendimientos equivalentes en este dataset, lo cual es esperado dado que Iris presenta clusters aproximadamente esféricos y sin ruido, el escenario ideal para ambos algoritmos. La ventaja del método jerárquico reside en que el dendrograma permite determinar el número de clusters sin necesidad de calcular silhouette o inercia para múltiples valores de K : la inspección visual del árbol es suficiente cuando los grupos naturales son bien definidos. DBSCAN, calibrado para Iris, detectó clusters aceptables pero con menor fidelidad respecto a las etiquetas reales, resultado esperado en datos con geometría esférica donde la densidad relativa no es el criterio más discriminante.

5.3 Reflexión

La selección del algoritmo de clustering no debe basarse en el nombre sino en las propiedades asumidas sobre los datos: esfericidad y número de clusters conocido favorecen K-Means; estructura de árbol e incertidumbre sobre K favorecen el jerárquico; presencia de ruido y clusters de forma arbitraria justifican DBSCAN. La reducción de dimensionalidad previa es en general beneficiosa para la estabilidad de los algoritmos de distancia, pero introduce una pérdida de información métrica global que debe tenerse en cuenta al interpretar el ARI.

6. Notebook 11: Interpretabilidad de Modelos

6.1 Consistencia de SHAP entre Random Forest y Gradient Boosting

El primer ejercicio extendió el análisis SHAP al `GradientBoostingClassifier` (100 estimadores) entrenado sobre el mismo dataset de cáncer de mama del cuerpo principal del notebook. Aplicando `shap.TreeExplainer` a ambos modelos, se compararon los *summary plots* y los rankings de importancia media por módulo del valor SHAP ($|\text{SHAP}|$).

Los modelos coincidieron en 4 de las 5 características más influyentes (principalmente *worst area*, *worst concave points*, *mean concave points* y *worst radius*), con diferencias solo en el orden relativo y en la magnitud de las contribuciones. Ambos alcanzaron una precisión similar en el conjunto de prueba (alrededor del 96.5%). Esta convergencia entre dos algoritmos con estrategias de construcción de árboles fundamentalmente distintas (Random Forest construye árboles independientes con submuestreo aleatorio; Gradient Boosting los construye secuencialmente minimizando residuos) refuerza la conclusión de que las características identificadas son genuinamente relevantes para la tarea, y no artefactos de una arquitectura específica.

6.2 Modelo surrogate para interpretabilidad eficiente de redes neuronales

El cuarto ejercicio abordó el problema práctico del costo computacional del `KernelExplainer` para redes neuronales. La solución implementada consistió en entrenar un árbol de decisión de profundidad máxima 4 (*surrogate model*) sobre las predicciones del MLP, y aplicar el rápido `TreeExplainer` sobre este árbol en lugar del costoso `KernelExplainer` sobre la red original.

El árbol surrogate alcanzó una *fidelidad* (fracción de predicciones del MLP reproducidas correctamente) de aproximadamente 0.938, indicando que modela fielmente el comportamiento de la red en casi el 94% de las instancias. La comparativa de tiempos fue contundente: el `TreeExplainer` sobre el surrogate completó la explicación de las 114 muestras del conjunto de prueba en aproximadamente 2 ms, frente a los más de 5 minutos requeridos por el `KernelExplainer` sobre solo 20 muestras. El *speedup* estimado supera las 1,000 veces. La comparativa de los 5 *features* más influyentes entre ambos métodos mostró una coincidencia de 4 de 5, lo que valida la confiabilidad del surrogate como aproximación interpretable para uso en producción.

6.3 Reflexión

La interpretabilidad de modelos complejos no requiere necesariamente métodos exactos y costosos. La estrategia del modelo surrogate –entrenar un modelo simple e interpretable que imite el comportamiento de uno complejo– es una técnica pragmática con sólido soporte teórico y empírico. Su viabilidad depende de que la fidelidad sea suficientemente alta; en caso contrario, el surrogate estaría describiendo su propio comportamiento, no el de la red original.

7. Notebook 12: CPU, GPU y Aceleración de Hardware

7.1 Impacto del *batch size* en el tiempo de entrenamiento

El segundo ejercicio realizó un barrido sistemático del *batch size* sobre cinco valores (32, 64, 128, 256, 512) para los modelos MLP y CNN entrenados durante 3 épocas en CPU. La Tabla 3 resume los tiempos observados.

Cuadro 3: Tiempo de entrenamiento (s) en CPU según *batch size* (3 épocas, MNIST).

<i>Batch size</i>	MLP (s)	CNN (s)
32	≈ 18,4	≈ 313
64	≈ 12,2	≈ 287
128	≈ 9,5	≈ 271
256	≈ 7,9	≈ 266
512	≈ 6,7	≈ 269

Para el MLP, la relación entre *batch size* y tiempo de entrenamiento es monótonamente decreciente en el rango evaluado: cada duplicación del tamaño de lote reduce el tiempo en aproximadamente un 20–25 %, resultado de la disminución del número de actualizaciones de pesos por época y del amortizamiento del overhead de Python por paso (*step*). Para la CNN, la reducción es más pronunciada en los valores bajos (de 32 a 128) y se aplana a partir de 256, donde el cuello de botella comienza a trasladarse de la frecuencia de actualización al cómputo de las convoluciones en sí. En GPU, el fenómeno es análogo pero con el punto de saturación desplazado hacia valores superiores (256–512), dado que la mayor cantidad de núcleos de cómputo paralelo puede aprovechar lotes más grandes antes de alcanzar la saturación.

7.2 Optimización del pipeline de datos con `tf.data`

El cuarto ejercicio comparó dos pipelines de carga de datos para 5 épocas de entrenamiento en CPU: un pipeline base (`Dataset.from_tensor_slices().batch()`) y un pipeline optimizado (`.cache().shuffle().batch().prefetch(AUTOTUNE)`). La mejora observada fue del orden del 18–22 % en tiempo total para el MLP y del 12–15 % para la CNN.

El mecanismo de `cache()` almacena el conjunto de datos completo en RAM tras la primera época, eliminando la lectura y deserialización del tensor en las épocas sucesivas.

El mecanismo `prefetch(AUTOTUNE)` superpone la preparación del siguiente lote de datos con la ejecución del paso de entrenamiento actual, eliminando el tiempo muerto de espera en la CPU mientras la GPU (o el núcleo de cómputo de la CPU) procesa el lote anterior. La mejora porcentual es moderada en CPU porque el cuello de botella principal es el cómputo de las operaciones matriciales, no la carga de datos. En GPU, donde el cómputo es más rápido, la proporción del tiempo dedicada a la carga de datos es mayor, y la mejora del pipeline puede llegar a ser de un factor $2\times$ o superior.

7.3 Reflexión

La optimización del rendimiento en Deep Learning debe atacarse en capas: primero el pipeline de datos (cuello de botella frecuentemente subestimado), luego el *batch size*, y finalmente la arquitectura del hardware. Un pipeline de datos ineficiente puede neutralizar completamente la ventaja de una GPU de última generación. La regla práctica es: antes de escalar hardware, medir si el cuello de botella está en el cómputo o en la I/O de datos.

8. Notebook 13: Despliegue de Modelos

8.1 Endpoint de predicción por lotes

El primer ejercicio implementó un endpoint `POST /predict/batch` que acepta una lista arbitraria de observaciones (validada mediante Pydantic) y devuelve una predicción con probabilidades por clase para cada una. La lógica del handler fue primero implementada y probada de forma directa (fuera del servidor HTTP) para validar la corrección del resultado, y luego integrada en el archivo `deploy/app.py`.

Un experimento de *benchmarking* comparó el tiempo de procesamiento de 100 predicciones mediante una única llamada batch frente a 100 llamadas individuales. La llamada batch completó el proceso en menos de 1 ms, mientras que el bucle de llamadas individuales requirió aproximadamente 8 ms, un *speedup* de $10\times$. En el contexto de una API HTTP real, la diferencia sería aún mayor: cada llamada individual carga el overhead de serialización HTTP, negociación de conexión TCP y deserialización de JSON, costos que en el caso batch se pagan una sola vez con independencia del número de observaciones. Para pipelines de inferencia a escala (cientos de miles de predicciones diarias), la diferencia entre endpoints batch e individuales puede traducirse en reducciones de latencia acumulada de varios órdenes de magnitud.

8.2 Contenedorización multi-etapa con Docker

El tercer ejercicio rediseñó el `Dockerfile` original usando una estrategia de construcción multi-etapa. La primera etapa (`AS deps`) usa `python:3.10-slim` como base e instala las dependencias con el prefijo `/packages`, sin incorporar los artefactos de la aplicación. La segunda etapa (`AS runtime`), también basada en `python:3.10-slim`, copia solo los paquetes instalados desde la primera etapa mediante `COPY --from=deps`, junto con el modelo serializado y el código de la API.

El beneficio principal de esta estrategia es la eliminación de `pip` y sus metadatos del contenedor de producción. Para la API de Iris con dependencias ligeras (scikit-learn, FastAPI, NumPy), el ahorro es de aproximadamente 20–25 MB. Para proyectos con dependencias de mayor peso (PyTorch, ≈ 800 MB; TensorFlow, ≈ 500 MB), la separación entre etapa de instalación y etapa de runtime puede reducir el tamaño final de la imagen en 200–500 MB, con impacto directo en los tiempos de descarga y despliegue en plataformas cloud. Adicionalmente, la imagen resultante no contiene herramientas de compilación ni gestores de paquetes, lo que reduce la superficie de ataque en términos de seguridad.

8.3 Reflexión

El despliegue de modelos en producción requiere el mismo rigor de diseño que la arquitectura del modelo. Un endpoint batch bien diseñado y un contenedor con capas mínimas no son optimizaciones cosméticas: son decisiones de ingeniería con impacto directo en el costo operativo, la latencia de respuesta y la seguridad del sistema. El ciclo completo entrenamiento – validación – API – contenedor – versionamiento constituye el mínimo viable para un sistema de ML reproducible y mantenible.

9. Conclusiones Generales

El recorrido por los notebooks 07 a 13 consolidó los aspectos más avanzados del ciclo de vida del aprendizaje profundo, desde la representación semántica del lenguaje hasta el despliegue en contenedores. A continuación se enuncian las lecciones transversales más relevantes de este tramo:

- 1. Los embeddings estáticos son una herramienta de producción viable.** La similitud coseno entre vectores GloVe reproduce relaciones semánticas comparables a las matrices de atención de los Transformers para tareas de proximidad temática, sin requerir infraestructura de inferencia costosa. La elección entre embeddings estáticos y modelos contextuales debe basarse en el requisito real de comprensión del contexto, no en la disponibilidad de la herramienta más moderna.
- 2. La arquitectura determina la representación, no solo el rendimiento.** La transición de una GAN densa a una DCGAN, o de un autoencoder denso a uno convolucional, no es solo una mejora cuantitativa en MSE o en calidad visual: refleja un cambio en el tipo de inducción estructural que el modelo puede explotar. Adaptar la arquitectura a la geometría del dominio del problema (espacial, temporal, semántica) es una decisión de diseño fundamental.
- 3. El espacio latente del VAE tiene propiedades verificables.** La interpolación en cuadrícula sobre el espacio latente 2D no es solo una visualización bonita: es una prueba empírica de que la regularización KL cumplió su función. Un espacio latente discontinuo o con regiones vacías indicaría que la red aprendió un autoencoder convencional disfrazado de VAE.
- 4. La reducción de dimensionalidad no preserva la métrica global.** t-SNE produce visualizaciones superiores a PCA para descubrir agrupamientos intrínsecos,

pero las coordenadas t-SNE no conservan las distancias del espacio original. K-Means aplicado sobre el espacio original de 64 dimensiones superó al aplicado sobre la proyección t-SNE en términos de ARI, lo que subraya que la elección del espacio de representación para clustering en producción debe priorizar la preservación de la geometría, no la separabilidad visual.

5. **La interpretabilidad tiene un costo computacional que puede gestionarse.** El modelo surrogate ofrece un equilibrio practicable entre exactitud explicativa y eficiencia: una fidelidad del 93 % con un *speedup* de $10^3\times$ frente al `KernelExplainer` lo convierte en la opción por defecto para entornos de producción donde se requieren explicaciones en tiempo real.
6. **El pipeline de datos es el cuello de botella oculto.** La optimización de `tf.data` con `cache` y `prefetch` mejora el rendimiento antes de invertir en hardware más potente. En GPU, donde el cómputo es más rápido, la proporción del tiempo de I/O es mayor, y el impacto del pipeline es más pronunciado.
7. **El despliegue es una disciplina de ingeniería, no un paso final trivial.** Un endpoint batch, un Dockerfile multi-etapa y un sistema de versionamiento de modelos no son adornos: son los mecanismos que convierten un experimento exitoso en un servicio confiable, reproducible y auditable.

Referencias

- Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proceedings of EMNLP*.
- Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., et al. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27.
- Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. *International Conference on Learning Representations (ICLR)*.
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.
- Documentación oficial de spaCy: <https://spacy.io/usage>
- Documentación oficial de Keras/TensorFlow: <https://keras.io/>
- Documentación oficial de scikit-learn: <https://scikit-learn.org/stable/>
- FastAPI Documentation: <https://fastapi.tiangolo.com/>
- Docker Multi-stage builds: <https://docs.docker.com/build/building/multi-stage/>
- TensorFlow `tf.data` Performance Guide: https://www.tensorflow.org/guide/data_performance